

基于 RUSTLINGS 题单整理 · RUST STUDY NOTES

Rust学习笔记

本笔记依据 Rustlings 题单系统整理 Rust 核心知识点：从变量、所有权与借用，到泛型、trait 与生命周期，再到迭代器、智能指针、并发、宏与 unsafe/FFI；并覆盖十道数据结构与算法强化题的完整思路解析。

涵盖 28 个专题 · 16 章笔记 · 3 个附录

目录

前言：这份笔记怎么用	4
第 1 章 初识 Rust：变量、类型与函数	4
1.1 变量绑定与可变性 (variables1-6)	4
1.2 基本类型 (primitive_types1-6)	5
1.3 函数：签名、表达式与返回值 (functions1-5)	6
1.4 if 是表达式 (if1-3)	6
1.5 打印与格式化 (intro2)	7
第 2 章 所有权：移动语义与借用	7
2.1 所有权三规则与移动语义 (move_semantics1-4)	7
2.2 引用与借用 (move_semantics5-6)	8
2.3 String 与 &str; (strings1-4)	8
2.4 Vec：可增长数组 (vecs1-2)	9
第 3 章 结构体与枚举	10
3.1 三种结构体与方法 (structs1-3)	10
3.2 枚举与模式匹配 (enums1-3)	11
第 4 章 模块系统与工程组织	11
4.1 私有与公有 (modules1)	11
4.2 use 与重导出 (modules2-3)	11
第 5 章 标准库集合：Vec 与 HashMap	12
5.1 HashMap 基本操作 (hashmaps1)	12
5.2 entry API：不存在才插入 (hashmaps2-3)	12
第 6 章 错误处理：Option 与 Result	13
6.1 Option (options1-3)	13
6.2 Result 与 ? 运算符 (errors1-3)	14
6.3 自定义错误与 map_err (errors4-6)	14
6.4 panic! 还是 Result?	15
第 7 章 泛型、Trait 与生命周期	15

7.1 泛型 (generics1-2)	15
7.2 Trait: 共享行为 (traits1-5)	16
7.3 生命周期 (lifetimes1-3)	16
7.4 标准库常用 trait 速览	18
第 8 章 类型转换	18
第 9 章 测试	19
9.1 断言与 should_panic (tests1-4)	19
9.2 构建脚本与条件编译 (tests7-8)	20
第 10 章 迭代器与函数式风格	20
10.1 Iterator 协议 (iterators1)	20
10.2 适配器与消费者 (iterators2-3)	21
10.3 fold 与函数式算法 (iterators4-5)	22
第 11 章 智能指针	22
11.1 Box: 堆分配与递归类型 (box1)	22
11.2 Rc 与 Arc: 引用计数 (rc1、arc1)	23
11.3 Cow: 写时克隆 (cow1)	23
第 12 章 并发编程	24
12.1 线程与 JoinHandle (threads1)	24
12.2 共享可变状态: Arc + Mutex (threads2)	24
12.3 消息传递: mpsc 通道 (threads3)	25
第 13 章 宏	25
第 14 章 unsafe 与 FFI	26
14.1 unsafe 的两种形态 (tests5)	26
14.2 裸指针与 Box 的互转 (tests6)	26
14.3 extern 块与符号导出 (tests9)	27
第 15 章 Clippy 与代码风格	27
第 16 章 强化训练: 数据结构与算法实战	28
16.1 链表: NonNull 与双链表反转 (algorithm1-2)	28
16.2 泛型排序 (algorithm3)	28

16.3 二叉搜索树 (algorithm4)	29
16.4 BFS 与 DFS (algorithm5-6)	29
16.5 栈与队列 (algorithm7-8)	30
16.6 二叉堆 (algorithm9)	30
16.7 无向图与 trait 抽象 (algorithm10)	31
附录 A 题单与知识点对照表	31
附录 B 常见编译错误速查	33
附录 C 延伸学习资源	34

前言：这份笔记怎么用

本笔记依据仓库中的 Rustlings 题单整理而成，覆盖题单全部 28 个专题、共约 100 余道练习背后的知识点。每章对应题单中的一组练习：先讲清核心概念，再用题单中的典型写法给出代码示例，最后附上易错点。建议的用法是：先读对应章节，再回去做（或重做）该组练习，用练习检验理解。

全书共 16 章：第 1-5 章是基础语法与所有权（对应题单 variables 到 hashmaps 专题）；第 6-9 章是类型系统与工程实践（options、error_handling、generics、traits、lifetimes、conversions、tests）；第 10-15 章是标准库进阶（iterators、smart_pointers、threads、macros、unsafe 与 FFI、clippy）；第 16 章针对题单末尾的 10 道算法强化题，把前面的知识落到数据结构实现上。附录 A 给出题单与知识点的完整对照表，附录 B 收录做题时最常见的编译错误速查，附录 C 是延伸学习资源。

贯穿全书的一条主线

Rust 的所有特性几乎都服务于同一个目标：**不依赖垃圾回收的内存与线程安全**。变量默认不可变、所有权与借用检查、Option/Result 显式处理空值与错误、Send/Sync 约束跨线程共享——每学一个特性，都可以问一句“它在为哪条安全保证服务”，知识点就会串成一张网。

第 1 章 初识 Rust：变量、类型与函数

对应题单：intro、variables、functions、if、primitive_types。这一章解决三个问题：变量怎么声明、有哪些内置类型、函数怎么定义与返回。

1.1 变量绑定与可变性 (variables1-6)

Rust 用 `let` 把值绑定到变量名。与多数语言不同，变量绑定**默认不可变 (immutable)**：想重新赋值必须写 `let mut`。这不是缺陷而是默认安全——可变性是你来要的，而不是默认送的。

```
// variables4: 没有 mut, 第二次赋值直接编译报错
fn main() {
    let x = 5;
    x = 6;           // error: cannot assign twice to immutable variable
    let mut y = 5;
    y = 6;           // OK
}
```

遮蔽 (Shadowing)：可以用同名 `let` 重新绑定一个变量，新绑定可以换类型。variables5 里把数字 3 转成字符串复用同一个名字，靠的就是遮蔽而非可变性：

```
fn main() {
    let x = 5;
    let x = x + 1;           // 遮蔽：同一作用域内重新绑定
    let x = x * 2;           // x == 12
    let spaces = "   ";
    let spaces = spaces.len(); // 类型从 &str 变为 usize，合法
}
```

常量 (const)：variables6 引出常量与变量的三点区别——用 `const` 声明、永远不可变（也不能加 `mut`）、类型标注不可省略；常量只能绑定常量表达式，可在任意作用域声明（包括全局）。

```
const MAX_POINTS: u32 = 100_000; // 必须标注类型，命名惯例全大写

fn main() {
    println!("max = {}", MAX_POINTS);
}
```

1.2 基本类型 (primitive_types1-6)

Rust 是静态强类型语言，编译期确定每个表达式的类型；但支持类型推断，多数场合不必手写。标量类型一览：

类型族	类型	说明 / 题单考点
整数	i8..i128, u8..u128, isize, usize	isize/usize 与平台指针宽度一致；字面量支持 10_000、0xff、0b1010、0o77 写法
浮点	f32, f64	默认推断为 f64；浮点数比较建议用容差而非 ==
布尔	bool	只有 true / false；if 条件必须是 bool，不接受隐式转换
字符	char	4 字节 Unicode 标量值，可存中文与 emoji；用单引号
单元	()	空元组即“无值”；不返回值的函数返回 ()

表 1-1 标量类型速览

复合类型有三种形态，primitive_types3-6 分别考察了数组初始化语法、切片、元组解构与索引：

- **数组** `[T; N]`：长度编译期固定。支持简写初始化 `["Are we there yet?"; 10]`，见 primitive_types3；
- **切片** `&[T]`：对数组/Vec 的引用视图，`&a[1..3]`，见 primitive_types4，区间可以省略端点：`&a[..3]`、`&a[3..]`
- **元组** `(T1, T2, ...)`：可聚合不同类型。支持解构 `let (name, age) = t;`，也支持索引 `t.0`，见 primitive_types5/6

```
fn main() {
    let a = [0; 100];           // 数组: 100 个 0
    let slice: &[i32] = &a[10..20]; // 切片: 借用其中一段
    let t: (&str, i32) = ("Alice", 30);
    let (name, age) = t;         // 解构
    println!("{}", slice.len(), name, t.1);
}
```

1.3 函数：签名、表达式与返回值 (functions1-5)

函数用 `fn` 定义。签名规则只有一条：**所有参数和返回值都必须有显式类型标注**，Rust 不对函数签名做跨函数推断（`functions2` 缺参数类型、`functions4` 缺返回类型都会被编译器点名）。

```
fn add(x: i32, y: i32) -> i32 { // 参数类型、返回类型都不可省略
    x + y                       // 末表达式即返回值，注意没有分号
}
```

表达式 vs 语句 (functions5 的核心)

Rust 区分**表达式 (expression)** 与**语句 (statement)**：表达式求值并产生值，语句执行动作返回 `()`。函数体最后一行若是表达式（**不带分号**），它就是返回值；多写一个分号就把它变成了语句，函数会返回 `()`——这正是 `functions5` “只删一个字符就能修好”的错误。想提前返回也可以写 `return num * num;`。

调用端同样受类型检查约束：`functions3` 里函数声明无参而调用传了参，`functions1` 则是调用了不存在的函数——读懂编译错误本身就是 Rust 学习的一部分。

1.4 if 是表达式 (if1-3)

Rust 的 `if` 不是语句而是**表达式**，可以直接用来产值，相当于其他语言的三元运算符。三条规则：条件不必加括号；条件必须是 `bool`；**所有分支必须返回同一类型**（`if2`、`if3` 都在考察这一点）。

```
fn max(a: i32, b: i32) -> i32 {
    if a > b { a } else { b } // if 表达式整体有值，两臂类型必须一致
}

fn main() {
    let n = 6;
    let parity = if n % 2 == 0 { "even" } else { "odd" };
    println!("{}", max(1, 2), parity);
}
```

循环方面题单没有专门练习，但后续大量出现：`loop` 无限循环可带 `break` 返回值；`while` 配条件；`for` 配迭代器（`for x in &v` 语法糖），是 Rust 里最常用的循环形式。

1.5 打印与格式化 (intro2)

`println!` 宏是第一个遇到的宏：格式化字符串里用 `{}` 占位，参数依次填充；常用形式还有 `{:?}` 调试打印、`{:#?}` 命名参数。宏调用必须带 `!`——它不是函数。

```
fn main() {
    let name = "Rust";
    println!("Hello, {}!", name);           // {} 占位
    println!("{0} vs {0}, {1}", name, 42);  // 位置参数
    println!("{lang}", lang = name);        // 命名参数
    let v = vec![1, 2];
    println!("{:?} {:?}", v, v);           // Debug 输出，多行更友好
}
```

第 2 章 所有权：移动语义与借用

对应题单：move_semantics、strings、vecs。所有权是 Rust 最核心也最独特的机制，题单用 6 道 move_semantics 练习层层推进：从“值被移走后不能再用”，到“何时该借用、何时该可变借用”。

2.1 所有权三规则与移动语义 (move_semantics1-4)

所有权三规则

1. Rust 中每个值都有一个**所有者 (owner)**；
2. 同一时刻只能有一个所有者；
3. 所有者离开作用域时，值被**丢弃 (drop)**，内存即时释放。

赋值、传参、返回值默认都是**移动 (move)**：所有权从旧的变量转移给新的变量，旧变量从此失效。move_semantics2 里把 `vec0` 传进 `fill_vec(vec0)` 后再使用 `vec0`，编译器报 "borrow of moved value"——不是数据没了，是所有权没了。

```
fn fill_vec(vec: Vec<i32>) -> Vec<i32> {    // 参数拿走所有权
    let mut vec = vec;
    vec.push(1);
    vec                                     // 所有权随返回值交还给调用者
}

fn main() {
    let vec0 = Vec::new();
    let vec1 = fill_vec(vec0);
    // println!("{:?}", vec0); // 报错: vec0 的值已被移走
    println!("{:?}", vec1);
}
```

move_semantics3/4 进一步练习改写函数形态：让参数直接以 `mut vec: Vec<i32>` 接收（省去函数内重新绑定），或让函数自己创建数据（调用端不再预建）。几种方案都合法，取舍标准是“谁需要持有这份数据”。

复制 (Copy) 语义是移动的例外：整数、浮点、bool、char 以及只含这些类型的元组/定长数组实现 Copy，赋值时按位复制，旧变量仍可用。**Vec** 不实现 Copy，所以 Vec/String 的赋值是移动。

2.2 引用与借用 (move_semantics5-6)

不想交出所有权时，用**引用**去“借”：不可变引用 **&T** 只读；可变引用 **&mut T** 可写。函数签名的选择口诀 (move_semantics6 的考点)：**只读用 &，要改用 &mut;，要拿走就直接传值。**

借用两规则 (编译器强制执行)

1. 同一时刻，对同一数据要么有**任意多个不可变引用**，要么有**恰好一个可变引用**，二者不能共存；
2. 引用必须始终有效——**悬垂引用**（指向已被释放的数据）无法通过编译。

move_semantics5 是理解“NLL（非词法作用域生命周期）”的经典练习：可变引用 **&mut x** 与不可变引用 **&x** 同时存活就会冲突，解决办法是让各引用的作用域不重叠——比如在打印（用不可变引用）之前先完成所有通过可变引用的写入：

```
fn main() {
    let mut x = 100;
    let y = &mut x;
    *y += 100;           // * 解引用，通过 y 写入 x
    let z = &x;          // y 的最后一次使用在上面的 +=，之后借用结束
    println!("x={x} z={z}"); // 600? 不，是 200 — 先改后读，互不重叠
}
```

解题时常遇到的另一个提示是“cannot borrow **vec** as mutable, as it is not declared as mutable”——错误在**绑定点**而不是使用点：声明处加上 **mut** 即可 (move_semantics1)。

2.3 String 与 &str (strings1-4)

Rust 字符串有两种核心形态，strings1/2 考察它们之间的转换：

	String	&str (字符串切片)
内存	堆上分配、可增长、拥有所有权	指向字节序列的引用 (borrowed)
创建	String::from("hi")、"hi".to_string()	字面量 "hi" ('static 生命周期)
修改	push_str / + / format! 可用	只读
典型角色	函数需要拥有/修改字符串时	函数只读字符串时的参数首选

表 2-1 String 与 &str 对比

```
fn current_favorite_color() -> String {
    String::from("blue")          // strings1: &str 转 String 的两种方式之一
    // "blue".to_string()         // 另一种
}

fn string_uppercase(data: &mut String) {    // strings 系列: 要修改就借 &mut
    data.to_uppercase();
}
```

Deref 强制转换 (deref coercion) : strings2 的"加一个字符"——在 `word` 后面加 `&`, 就能把 `String` 自动转换为 `&str`; 传给接收切片的函数。规则是: `&String`; 传给需要 `&str`; 的位置时编译器自动插入转换, 这让大家平时几乎不用手写 `&word[..]`。

strings3 练习常用字符串方法: `trim()` 去首尾空白、`split()` 按分隔符切割、`format!` 拼接新串 (比 `+` 更灵活, 不消耗操作数)。注意这些方法大多**返回新值**而不是原地修改——`String` 的方法分原地 (`push_str`) 与复制 (`trim`、`to_uppercase`) 两族。

```
fn compose_me(input: &str) -> String {
    format!("{}", world", input)    // format! 生成新的 String
}
```

2.4 Vec: 可增长数组 (vecs1-2)

vecs1 考察两种建 `Vec` 的方式; vecs2 考察两种遍历修改的方式。再次提醒: **把 `Vec` 传值给函数会拿走所有权**, 遍历时 `iter()` 借用、`into_iter()` 拿走元素、`iter_mut()` 可变借用——三种迭代器对应三种访问级别 (第 10 章展开)。

```
fn main() {
    let mut v: Vec<i32> = Vec::new();    // 方式一: new + push
    v.push(1);
    let v2 = vec![1, 2, 3];             // 方式二: vec! 宏, 更常用

    for element in &mut v2 {             // vecs2 写法一: 可变引用 + 解引用
        *element *= 2;
    }
    for element in v2.iter_mut() {       // 写法二: 显式 iter_mut, 等价
        *element += 1;
    }
}
```

第 3 章 结构体与枚举

对应题单：structs、enums。结构体把相关数据打包命名，枚举把“一个值可能是几种形态”类型化——再配合模式匹配，是 Rust 表达业务模型的主力工具。

3.1 三种结构体与方法 (structs1-3)

形态	定义	适用场景
经典结构体	<code>struct P { name: String, age: u32 }</code>	命名字段，最常用
元组结构体	<code>struct Point(i32, i32);</code>	给元组起名、区分同构类型
单元结构体	<code>struct Marker;</code>	无字段，用于 trait 占位/类型标记

表 3-1 三种结构体 (structs1 要求各实现一个)

实例化的两个语法糖：**字段初始化缩写**（变量名与字段同名时只写一次）与**struct 更新语法** `..p1`（用已有实例填充剩余字段）——structs2 的考点。注意更新语法对未指定的字段是**移动**，原实例中被移走的字段不能再整体使用。

```
struct Package {
    sender_country: String,
    recipient_country: String,
    weight_grams: i32,
}

impl Package {
    // 关联函数（无 self）：类似其他语言的“静态方法”，常用作构造器
    fn new(sender: String, recipient: String, weight: i32) -> Package {
        Package { sender_country: sender, recipient_country: recipient,
            weight_grams: weight }
    }
    // 方法：第一个参数 &self，表示借用实例
    fn is_international(&self) -> bool {
        self.sender_country != self.recipient_country
    }
    // 需要读 self 且接收额外参数的方法 (structs3 的 get_fees)
    fn get_fees(&self, cents_per_gram: i32) -> i32 {
        self.weight_grams * cents_per_gram
    }
}
```

`impl` 与 `trait` 的区别是理解方法系统的关键：前者定义类型的行为，后者定义共享行为的接口，见第 7 章。方法选择器 `.` 自动解引用，`&p` 与 `p` 都能调到方法。

3.2 枚举与模式匹配 (enums1-3)

枚举的每个变体 (variant) 可以携带不同形态的数据：无数据、匿名结构体、元组、单个值等。enums2 要求把这些形态都写出来。这让枚举成为"和类型" (sum type)，配合 match 做穷尽处理：

```
enum Message {
    Quit,                // 无数据
    Move { x: i32, y: i32 }, // 匿名结构体
    Write(String),        // 单值
    ChangeColor(u8, u8, u8), // 元组
}

impl Message {
    fn process(&self) {
        match self {
            // enums3: match 解构每个变体
            Message::Quit => println!("quit"),
            Message::Move { x, y } => println!("move to {x},{y}"),
            Message::Write(text) => println!("write {text}"),
            Message::ChangeColor(r, g, b) => println!("rgb({r},{g},{b})"),
        }
    }
}
```

match 的两条铁律

1. **穷尽性**：必须覆盖所有可能的变体，漏一个编译不过（这是特性不是负担）；
2. **模式可以绑定值**：如 Write(text) 把变体内的 String 绑定到 text 上，还能用 _ 作为"其余全部"的通配分支。

第 4 章 模块系统与工程组织

对应题单：modules。Rust 的可见性默认全部私有，模块系统解决"代码怎么拆文件、名字怎么引入"两件事。

4.1 私有与公有 (modules1)

条目（函数、结构体、字段、模块）默认对外不可见，加 pub 才公开。modules1 的修复就是给 eat_at_restaurant 加上 pub。注意：**结构体的字段是独立可见的**——结构体公开但字段私有时，外部能拿到类型却不能构造/读写字段。

4.2 use 与重导出 (modules2-3)

use 把路径引入当前作用域，三种写法（modules3 考察后两种）：

```
use std::time::{SystemTime, UNIX_EPOCH}; // 嵌套路径：一行引多个
use std::collections::*;                  // glob: 全部引入（慎用）

pub use crate::front_of_house::fish as tasty_fish; // pub use 重导出
// modules2: 内部叫 fruits::APPLE, 对外暴露成 delicious_snacks::fruit 成功
```

重导出 (pub use) 让模块对外呈现的接口与内部结构解耦——内部按实现分层组织，对外按使用方视角命名。这是库 API 设计的常用手段。

第 5 章 标准库集合：Vec 与 HashMap

对应题单：vecs、hashmaps。集合是泛型的第一个大规模应用场景，题单重点考 HashMap 的 entry API。

5.1 HashMap 基本操作 (hashmaps1)

```
use std::collections::HashMap;

fn main() {
    let mut basket: HashMap<String, u32> = HashMap::new();
    basket.insert(String::from("apple"), 3); // key/value 被移动进 Map
    let n = basket.get("apple");             // 返回 Option<u32>
    println!("{:?}", n);                     // Some(3)
}
```

注意 `get` 返回 `Option<V>`——键可能不存在，集合查询天然与 `Option` 绑定（第 6 章）。另外 `HashMap` 的键要求实现 `Eq + Hash`，值要求 `Sized`。

5.2 entry API：不存在才插入 (hashmaps2-3)

`entry(key)` 返回一个枚举：键存在得到 `Occupied`（内含可变引用），不存在得到 `Vacant`（内含占位）。`.or_insert(0)` 组合出“没有就插入默认值，有就拿到引用”的原子操作，是 `hashmaps2/3` 的标准解法：

```
// hashmaps3: 统计每队得分并累加
let mut scores: HashMap<String, i32> = HashMap::new();
for (team, points) in [("blue", 10), ("red", 50), ("blue", 25)] {
    let entry = scores.entry(team.to_string()).or_insert(0);
    *entry += points; // or_insert 返回 &mut V, 解引用后累加
}
```

集合	结构	典型用途 / 题单关联
<code>Vec<T></code>	连续可增长数组	顺序表；vecs、algorithm7 栈

集合	结构	典型用途 / 题单关联
VecDeque<T>	双端队列	队首队尾 $O(1)$; algorithm5 BFS、algorithm8
HashMap<K, V>	哈希表（无序）	键值统计; hashmaps、algorithm10 邻接表
HashSet<T>	哈希集合	去重与成员查询; algorithm6 DFS 的 visited
BTreeMap<K, V>	有序映射（B 树）	需要按键有序遍历时替换 HashMap

表 5-1 常用集合速览

第 6 章 错误处理：Option 与 Result

对应题单：options、error_handling。Rust 没有 null 和异常：“可能没有值”用 Option 类型化，“可能失败”用 Result 类型化，调用方被迫在类型系统层面面对这两种情况。

6.1 Option<T> (options1-3)

Option<T> 只有两个变体：Some(v) 与 None。取内部值的方式决定你的安全等级：

- `unwrap()`：直接取值，None 时 panic——测试与原型可用，生产代码是隐患；
- `expect("msg")`：同上但带自定义错误信息，崩得明白一点；
- `match / if let / while let`：安全处理，编译器保证你考虑了 None。

```
// options2: if let / while let 处理（可嵌套 Some(Some(x)))
fn main() {
    let nested: Option<Option<i32>> = Some(Some(7));
    if let Some(inner) = nested {
        if let Some(v) = inner {
            println!("value = {v}");
        }
    }
    // Option::flatten 可把 Option<Option<T>> 压平成 Option<T>

    let mut stack = vec![Some(1), None, Some(3)];
    while let Some(top) = stack.pop() { // 栈空时 pop 返回 None，循环结束
        if let Some(v) = top { println!("{v}"); }
    }
}
```

options3 的坑：**match 会取得枚举的所有权**。若只想借用内部值，在模式里写 `ref`（如 `Some(ref value)`）即可按引用绑定，避免 partial move。

6.2 Result<T, E> 与 ? 运算符 (errors1-3)

`Result<T, E>` 是"可能失败"的版本：`Ok(v)` 携带成功值，`Err(e)` 携带错误。`errors1` 要求把返回 `Option` 的函数改造成 `Result`——两者结构同构，语义不同：`None` 是"合法的无"，`Err` 是"失败的因"。

```
fn generate_nametag_text(age: i32) -> Result<String, String> {
    if age > 18 {
        Ok(format!("Hi! I am {age}"))
    } else {
        Err(String::from("age must be greater than 18"))
    }
}
```

? 运算符：错误传播的语法糖 (errors2)

表达式 `%s` 展开等价于：如果是 `Ok`，取出值继续；如果是 `Err`，**立即从当前函数 return Err(转换后的错误)**。它的隐藏动作是调用 `%s` 做错误类型转换（第 6.4 节），所以要求所在函数的返回错误类型与表达式错误类型满足 `From` 关系。

```
fn total_cost(item_quantity: &str) -> Result<i32, ParseIntError> {
    let quantity: i32 = item_quantity.trim().parse()?; // errors2: 用 ? 传播
    Ok(quantity * 10)
}
```

`errors3` 指出 `main` 也能返回 `Result`——惯例是 `Result<(), E>`：成功无值（单元类型），失败携带错误，配合 `?` 让 `main` 成为错误传播链的终点。

6.3 自定义错误与 map_err (errors4-6)

`errors4` 练习"校验构造函数"模式：构造可能失败时返回 `Result`，把非法输入挡在类型之外（无效状态不可表示）。`errors5` 解决多错误类型并存：用 `Box<dyn Error>` 装箱——所有实现了 `std::error::Error` 的错误都能塞进去，`?` 的 `From` 转换会自动完成装箱。

```
use std::error::Error;
use std::fmt;

#[derive(Debug)]
struct ParsePosNonzeroError { kind: ParseErrorKind }
impl fmt::Display for ParsePosNonzeroError { /* ... */ }
impl Error for ParsePosNonzeroError {}

fn parse_pos_nonzero(s: &str)
    -> Result<PositiveNonzeroInteger, ParsePosNonzeroError> {
    // errors6: map_err 把底层解析错误包装成自定义错误，再交给 ? 传播
    let num: i32 = s.parse().map_err(ParsePosNonzeroError::from_parse)?;
    PositiveNonzeroInteger::new(num).map_err(ParsePosNonzeroError::from_validation)
}
```

`map_err` 只改变错误的类型、保留成功路径，是错误分层（底层错误翻译成业务错误）的标准工具。错误类型的完整实现要满足三件套：`Debug` 派生、`Display` 展示、`impl Error` 空实现标记。

6.4 panic! 还是 Result?

	panic! (不可恢复)	Result (可恢复)
语义	程序进入不可信状态，立即终止	失败是预期内的结果之一
适用	bug、断言、演示代码	用户输入、文件/网络 IO、解析
调用方	无法拦截	必须处理（或继续传播）
题单对应	tests4 的 <code>should_panic</code>	errors1-6 全部

表 6-1 两种错误通道

第 7 章 泛型、Trait 与生命周期

对应题单：generics、traits、lifetimes、quiz3。三者常合称“泛型三部曲”：泛型让类型成为参数，trait 约束这些参数的能力，生命周期约束引用的有效范围。

7.1 泛型 (generics1-2)

泛型把“对任意类型都成立”的逻辑参数化。题单两题分别考：泛型集合需要明确元素类型，`generics1` 的 `Vec` 要知道推什么；以及把具体类型泛化，`generics2` 把 `Wrapper<u32>` 改成 `Wrapper<T>`：

```
struct Wrapper<T> {
    value: T,
}

impl<T> Wrapper<T> {           // impl 后也要声明 <T>
    fn new(value: T) -> Self { Self { value } }
}

// 在方法中只为特定类型实现（理解即可）
impl Wrapper<f64> {
    fn as_f64(&self) -> f64 { self.value }
}
```

泛型是**零成本抽象**：编译器为每个实际用到的具体类型生成专门代码（单态化），运行时没有虚表查找。代价是编译产物变大、编译变慢。

7.2 Trait: 共享行为 (traits1-5)

trait 定义一组方法签名，类型通过 `impl Trait for Type` 提供实现。traits1/2 考察基本定义与接收所有权 `self` 的实现；traits3 考察**默认实现**——实现者可以覆写，也可以直接用，默认方法还能调用同 trait 的其他方法：

```
trait AppendBar {
    fn append_bar(self) -> Self;           // traits2: 拿走 self 返回 Self
}

impl AppendBar for Vec<String> {
    fn append_bar(mut self) -> Self {
        self.push(String::from("bar"));    // 修改后交回所有权
        self
    }
}

trait Pilot {
    fn fly(&self);
    fn name(&self) -> String {              // traits3: 默认实现
        String::from("default pilot")
    }
}
```

trait 作为参数有三种写法 (traits4/5 考察)：

写法	示例	说明
<code>impl Trait</code>	<code>fn f(x: impl Display)</code>	简洁；适合单个参数
<code>trait bound</code>	<code>fn f<T: Display>(x: T)</code>	泛型显式声明；便于多参数同型
多重约束 +	<code>fn f(x: impl Display + Debug)</code>	要求同时实现多个 trait
where 子句	<code>fn f<T>(x: T) where T: Display + Debug</code>	约束多时更可读

表 7-2 trait 约束的写法 (quiz3 用 where + Display)

quiz3 综合练习 trait bound：给任意 `Display` 的类型实现 `OutlinedPrint` 并打印，需要 `use std::fmt::Display`；引入标准库的 `Display`——三种写法在这里可以互换。

7.3 生命周期 (lifetimes1-3)

生命周期标注不改变任何引用的实际寿命，它只是**向编译器描述引用之间的约束关系**，帮编译器验证没有悬垂引用。lifetimes1 要求标注 `fn longest<'a>(x: &'a str, y: &'a str) -> &'a str`：

```
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str { // 返回值活不过两个输入
    if x.len() > y.len() { x } else { y }
}

fn main() {
    let string1 = String::from("long string");
    let result;
    {
        let string2 = String::from("xyz");
        result = longest(string1.as_str(), string2.as_str());
        println!("{result}"); // 使用点必须落在两个输入都还活着的范围内
    } // lifetimes2: 把 println 移进块内, 或延长 string2
}
```

省略规则（为什么平时不用写 'a）

1. 每个引用参数各自获得独立生命周期；
 2. 只有一个输入引用时，它赋给全部输出引用；
 3. 含 &self; / &mut; self 的方法，self 的生命周期赋给全部输出。
- 三条都不满足（如 lifetimes1 的两入一出）时必须手写标注。

lifetimes3 考察**结构体持有引用**：字段含引用时，结构体定义与 impl 都要声明生命周期，含义是"结构体实例不得活过它借用的数据"：

```
struct Excerpt<'a> {
    part: &'a str,
}

impl<'a> Excerpt<'a> {
    fn announce(&self) -> &str { // 按省略规则 3, 返回值取 &self 的生命周期
        self.part
    }
}
```

补充两个常见标记：'static 表示"整个程序运行期间存活"（字符串字面量）；'_ 表示"比入参活得久"，常见于返回新建的迭代器/闭包。

7.4 标准库常用 trait 速览

trait	作用	常见触发场景
Display / Debug	用户可读 / 调试输出	println! 的 {} 与 {:?}
Clone / Copy	显式克隆 / 隐式复制	.clone()、赋值语义
PartialEq / Eq	相等比较	== 与 assert_eq!
Ord / PartialOrd	全序 / 部分序	sort()、max()
Hash	可哈希	作 HashMap 的键
Default	默认值	Default::default()、#[derive(Default)]
Iterator	迭代器协议	for 循环、迭代适配器（第 10 章）
From / Into	无损转换	String::from、.into()（第 8 章）

表 7-3 高频标准 trait（多可用 #[derive] 派生）

第 8 章 类型转换

对应题单：conversions（5 题）。Rust 几乎不做隐式转换，转换必须显式声明意图，按"会不会失败"分成两族。

机制	方向	失败语义	题单
as 强制转换	基本类型之间	截断/回绕，不报错	using_as
From / Into	T -> U（无损）	不失败	from_into
TryFrom / TryInto	T -> U（需校验）	返回 Result	try_from_into
FromStr	&str -> T	返回 Result	from_str
AsRef / AsMut	借用视图转换	不失败	as_ref_mut

表 8-1 转换 trait 一览

```
// as: 简单粗暴，注意精度丢失（using_as 求平均数要先把和转成 f64）
let a = 7_i32;
let b = a as f64 / 2.0;

// From: 实现 From 自动获得 Into (blanket impl)
impl From<(&str, i32)> for Person {
    fn from((name, age): (&str, i32)) -> Person {
        Person { name: String::from(name), age }
    }
}
let p: Person = ("Alice", 30).into();
```

```
// FromStr: 让 "30" 这样 str 能 parse 出自定义类型 (配合 .parse())
impl FromStr for Person {
    type Err = ParsePersonError;
    fn from_str(s: &str) -> Result<Person, Self::Err> { /* ... */ }
}

// TryFrom: 需要校验范围的转换, 返回 Result
impl TryFrom<i32> for PositiveNonzeroInteger {
    type Error = CreationError;
    fn try_from(value: i32) -> Result<Self, Self::Error> {
        PositiveNonzeroInteger::new(value) // 复用校验构造
    }
}

// AsRef: 泛型接收"任何能借出 &str 的东西"
fn word_count<S: AsRef<str>>(s: S) -> usize { s.as_ref().split_whitespace().count() }
// String、&str、&String 都能直接传入
```

两个经验法则：**优先实现 From 而不是 Into**（Into 会自动获得）；**API 参数用 impl AsRef<str> 或 AsMut<u32> 泛型**，调用方传 String 还是 &str; 都不用转换——这也是 as_ref_mut 一题的教学意图。

第 9 章 测试

对应题单：tests1-9。Rust 原生内置测试框架，无需第三方库：标注 `#[test]` 的函数就是测试，`cargo test` 运行。

9.1 断言与 should_panic (tests1-4)

```
#[cfg(test)]
mod tests { // 测试模块惯例: cfg(test) 使其不进发布产物
    use super::*;

    #[test]
    fn it_works() {
        assert!(true); // tests1: 断言为真
        assert_eq!(2 + 2, 4); // tests2: 断言相等 (顺序: 实际, 期望均可)
        assert_ne!(3, 4); // 断言不等
    }

    #[test]
    fn fun() {
        assert!(is_odd(4)); // tests3: 断言里可以调用函数
    }

    #[test]
    #[should_panic] // tests4: 期望函数 panic 才算通过
}
```

```
fn negative_rect() {
    Rectangle::new(-10, -20);    // 用负数触发 panic
}
}
```

- 断言可以追加自定义消息: `assert!(cond, "msg {x}")`;
- `#[should_panic]` 让"预期 panic"成为正向断言, 可加 `expected = "width"` 匹配 panic 消息子串;
- `debug_assert!` 在发布构建中完全编译掉, 零运行时开销;
- 测试函数签名也可以返回 `Result<(), E>`——测试内部也能用? 传播错误。

9.2 构建脚本与条件编译 (tests7-8)

题单强化练习 tests7/8 涉及 `build.rs`: 放在包根目录的构建脚本, 在编译前由 Cargo 执行, 用特殊格式的 `println!` 输出向 Cargo 下指令:

```
// build.rs
fn main() {
    println!("cargo:rustc-env=TEST_F00=42");    // 设置编译期环境变量
    println!("cargo:rustc-cfg=pass");           // 启用 cfg(feature = "pass")
}
```

```
// 代码里读取:
fn main() {
    let v: u32 = env!("TEST_F00").parse().unwrap(); // env! 宏读编译期变量
    #[cfg(feature = "pass")]
    println!("cfg pass is enabled");
}
```

第 10 章 迭代器与函数式风格

对应题单: iterators (5 题)。迭代器是 Rust 函数式风格的核心: 惰性求值、零成本抽象, 配合适配器链能替代大多数手写循环。

10.1 Iterator 协议 (iterators1)

实现 `Iterator` 只需一个方法 `next()`: 返回 `Option<Self::Item>`, 有下一个值给 `Some`, 耗尽给 `None`。集合不直接实现 `Iterator`, 而是提供三个转换方法:

方法	元素类型	所有权
<code>iter()</code>	<code>&T</code>	借用集合, 之后集合仍可用
<code>iter_mut()</code>	<code>&mut T</code>	可变借用, 可原地修改

方法	元素类型	所有权
into_iter()	T	拿走所有权，集合被消耗

表 10-1 三种获得迭代器的方式

```

let fruits = ["banana", "apple", "coconut"];
for fruit in fruits.iter() {           // iterators1 第一步：先变成迭代器
    println!("{}", fruit);
}
let mut it = fruits.iter();
assert_eq!(it.next(), Some(&"banana")); // 手动驱动：next 返回 Option
assert_eq!(it.next(), Some(&"apple"));
// 迭代器耗尽后 next 永远返回 None

```

10.2 适配器与消费者 (iterators2-3)

迭代器方法分两族：**迭代适配器** (lazy, 返回新迭代器：map、filter、enumerate、zip) 与**消费适配器** (驱动求值：sum、collect、fold、count、any、all、max)。适配器链必须以消费者结尾，否则什么都不执行。

```

// iterators2: 首字母大写 — char 方法 + collect 聚合
fn capitalize_first(input: &str) -> String {
    let mut c = input.chars();
    match c.next() {
        None => String::new(),
        Some(first) => first.to_uppercase().collect::<String>() + c.as_str(),
    }
}

let words: Vec<String> = vec!["hello", "world"]
    .iter()
    .map(|w| capitalize_first(w)) // 适配：惰性
    .collect();                  // 消费：这里才真正执行

// iterators3: Result 与 collect 的两种聚合方向
let results: Vec<Result<i32, DivisionError>> = ...;
let single: Result<Vec<i32>, _> = results.into_iter().collect(); // 任一 Err -> Err
let list: Vec<Result<i32, _>> = results;                          // 保留每个结果

```

collect 的类型魔法

`collect()` 的结果类型由**调用处的标注或上下文推断**决定：同一个迭代器可以 collect 成 Vec、String、HashMap (键值对)、甚至 Result, E>。这就是 iterators3 里“一个函数改个返回类型、逻辑不变”的原因，背后是 FromIterator trait 在按目标类型收集元素。

10.3 fold 与函数式算法 (iterators4-5)

```
// iterators4: 阶乘 — range + fold, 无需可变量
fn factorial(num: u64) -> u64 {
    (1..=num).fold(1, |acc, x| acc * x)    // (初始值, 累计函数)
}

// iterators5: 对 HashMap 切片做统计 — fold 累计 + 方法链
fn count_collection_iterator(collection: &[HashMap<String, Progress>]) -> u32 {
    collection.iter()
        .flat_map(|map| map.values())      // 多个集合拍平成一个流
        .map(|p| match p { Progress::None => 0, Progress::Some(v) => *v })
        .sum()                             // 或 fold(0, |acc, v| acc + v)
}
```

常用组合子速记：**map** 转换、**filter** 过滤、**enumerate** 带序号、**flat_map** 拍平嵌套、**fold** 折叠、**sum** 求和、**zip** 逐对合并、**take** 取前 n 个、**skip** 跳过、**group_by** 分组收集。

第 11 章 智能指针

对应题单：smart_pointers (box1、rc1、arc1、cow1)。普通引用只是借用；智能指针**拥有**数据并提供额外能力（堆分配、引用计数、写时克隆）。

11.1 Box<T>：堆分配与递归类型 (box1)

Box<T> 把数据放到堆上，栈上只留一个指针。最重要的用途是**递归类型**：编译器必须知道类型大小，而链表节点“包含自己”会无限递归下去——Box 的大小固定（一个指针宽），recursion 到此为止：

```
enum List {
    Cons(i32, Box<List>),    // 不加 Box 编译器报"递归类型大小无法确定"
    Nil,
}

use List::{Cons, Nil};

fn main() {
    let list = Cons(1, Box::new(Cons(2, Box::new(Cons(3, Box::new(Nil))))));
}
```

Box 还实现 **Deref**，使 **Box<String>** 的智能指针能像普通引用一样使用方法——第 2 章 deref 强制转换的底层机制正是它。

11.2 Rc<T> 与 Arc<T>: 引用计数 (rc1、arc1)

所有权规则"唯一所有者"在图、树这类**多处共享**的结构上太苛刻。`Rc<T>` 用引用计数允许多个所有者：`Rc::clone` 只克隆计数不克隆数据，最后一个引用销毁时数据才释放。`rc1` 的太阳-行星模型是标准演示：

```
use std::rc::Rc;

let sun = Rc::new("Sun".to_string());
let earth = Rc::clone(&sun);           // 引用计数 +1, 不复制字符串
let mars = Rc::clone(&sun);
println!("count = {}", Rc::strong_count(&sun)); // 3
drop(mars);                             // 显式销毁一个引用
println!("count = {}", Rc::strong_count(&sun)); // 2
```

`Arc<T>` 是线程安全版（Atomic 引用计数），`Rc<T>` 则不能跨线程（未实现 Send）。两者内部都不可变——要共享**可变**状态需配合内部可变性类型（`RefCell<T>`）或转向 `Mutex<T>`（第 12 章）。

11.3 Cow<T>: 写时克隆 (cow1)

`Cow<B, ToOwned>`（clone on write）在"多数情况只读、偶尔需要修改"时避免不必要的复制：持有借用直到 `to_mut()` 被调用才真正克隆。`cow1` 的考点：**Borrowed 状态下 `to_mut` 不克隆，Owned 状态下原样返回。**

```
use std::borrow::Cow;

fn abs_all(input: &mut Cow<[i32]>) {
    for i in 0..input.len() {
        let v = input[i];
        if v < 0 {
            input.to_mut()[i] = -v; // 只有真的要改时才发生克隆
        }
    }
}
```

指针	语义	典型场景
<code>Box<T></code>	独占所有权，堆分配	递归类型、大对象转移
<code>Rc<T></code>	共享所有权（单线程）	树/图的多个父节点
<code>Arc<T></code>	共享所有权（多线程）	线程间共享只读数据
<code>Cow<T></code>	读借用/写克隆二选一	避免多数只读场景的复制
<code>RefCell<T></code>	内部可变性（运行期检查）	运行时借用检查（题单未直接考）

表 11-1 智能指针选型

第 12 章 并发编程

对应题单：threads（3 题）。Rust 的口号“无畏并发”建立在编译期检查之上：所有权与 trait 约束（Send/Sync）让数据竞争在编译阶段暴露。

12.1 线程与 JoinHandle (threads1)

`thread::spawn` 创建新线程，闭包捕获它需要的变量（捕获即移动，除非引用）。返回的 `JoinHandle<T>` 用 `.join()` 等待线程结束并取回返回值——主线程不 join 就退出的话，子线程会被直接杀掉：

```
use std::thread;

fn main() {
    let handles: Vec<_> = (0..10).map(|i| {
        thread::spawn(move || {           // move 把 i 的所有权交给闭包
            sleep_ms(100);
            i * 2                           // 线程返回值由 join 取回
        })
    }).collect();

    let results: Vec<i32> = handles.into_iter()
        .map(|h| h.join().unwrap())        // threads1: 逐个 join 收集结果
        .collect();
}
```

12.2 共享可变状态: Arc + Mutex (threads2)

多线程共享**可变**数据的组合拳：`Arc` 提供跨线程共享所有权，`Mutex<T>` 保证同一时刻只有一个线程能拿到数据。`.lock()` 返回守卫（guard），读写在它身上进行，离开作用域自动解锁：

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    let status = Arc::new(Mutex::new(JobStatus { jobs_completed: 0 }));
    let mut handles = vec![];
    for _ in 0..10 {
        let status = Arc::clone(&status);           // 计数 +1, 不复制数据
        handles.push(thread::spawn(move || {
            let mut locked = status.lock().unwrap(); // 拿锁 (RAII 守卫)
            locked.jobs_completed += 1;
        }));    // locked 在此离开作用域，锁自动释放——别把 sleep 括进来！
    }
    for h in handles { h.join().unwrap(); }
    println!("{}", status.lock().unwrap().jobs_completed);
}
```

threads2 的经典坑：锁的粒度

如果把 `.lock()` 写在睡眠/长计算的外面（长时间持有守卫），其他线程全部阻塞等锁，并发退化成串行。
守卫的作用域就是临界区——需要时用块 `{ let g = m.lock()...; }` 显式收窄。

12.3 消息传递：mpsc 通道 (threads3)

另一条思路是"通过通信共享内存"：`std::sync::mpsc::channel()` 创建多生产者单消费者通道。
 发送端 `Sender` 可 clone 出多个；接收端 `Receiver` 迭代接收，所有发送端 drop 后循环自然结束：

```
use std::sync::mpsc;
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();
    for i in 0..5 {
        let tx = tx.clone();           // 每个线程一个发送端
        thread::spawn(move || { tx.send(i).unwrap(); });
    }
    drop(tx);                          // 主线程的发送端关闭
    for received in rx {               // Receiver 实现了 Iterator
        println!("got {received}");
    }
}
```

第 13 章 宏

对应题单：macros（4 题）。宏在编译期展开成代码，能做函数做不到的事：可变参数，例如 `println!("a={ } b={ }", a, b)`；以及编译期检查格式串与参数匹配、控制语法结构。

```
// macros1: 调用宏必须带感叹号
my_macro!();

// macros2: 宏必须先定义后使用（与函数不同）
macro_rules! my_macro {
    () => {
        println!("Check out my macro!");
    };
}

// macros3: 跨模块导出宏要加 #[macro_export]
#[macro_export]
macro_rules! my_macro { () => { println!("exported!"); }; }

// macros4: 多个 arm 之间用分号分隔
macro_rules! my_macro {
    () => { println!("empty"); };
    ($val:expr) => { println!("val = {}", $val); }; // 接收一个表达式
}
```

}

声明宏的语法要点：模式（如 `$val:expr`）与展开模板成对出现，arm 之间分号分隔；片段说明符有 `expr`（表达式）、`ident`（标识符）、`ty`（类型）、`lit`（字面量）等；重复语法 `$(...),*` 处理可变参数。题单只覆盖声明宏，过程宏（proc-macro）属于进阶话题，见附录 C。

第 14 章 unsafe 与 FFI

对应题单：tests5、tests6、tests9（强化训练部分）。unsafe 不是关闭检查的“后门”，而是**把一部分安全责任从编译器移交给你的**契约机制。

14.1 unsafe 的两种形态 (tests5)

形态一：标记在**条目**上（unsafe fn、unsafe trait）——声明“调用/实现我需要满足额外契约”，契约写在 `# Safety` 文档注释里；形态二：标记在**代码块**上——声明“我在此手动维护了契约”，并在块前用 `SAFETY:` 注释说明依据。两种形态都必须配合文档，否则审查无法进行：

```
/// # Safety
///
/// `address` 必须是一个指向有效 u32 的可变引用的地址。
unsafe fn modify_by_address(address: usize) {
    // SAFETY: 调用方按契约保证 address 指向有效 u32。
    unsafe {
        let ptr = address as *mut u32;
        *ptr += 1;
    }
}
```

14.2 裸指针与 Box 的互转 (tests6)

`Box::into_raw` 交出所有权得到裸指针，`Box::from_raw` 从裸指针重建 Box。这对函数用于 FFI 边界或手动内存管理，重建后所有权回到 Rust 体系，drop 时机重新确定——**严禁对同一指针 from_raw 两次**（双重释放）：

```
let boxed = Box::new(Foo { a: 1, b: None });
let raw: *mut Foo = Box::into_raw(boxed); // 所有权移出 Rust 世界
unsafe {
    let rebuilt: Box<Foo> = Box::from_raw(raw); // 重建所有权
    // 使用 rebuilt ...
} // rebuilt 在此 drop，内存恰好释放一次
```

14.3 extern 块与符号导出 (tests9)

`extern` 块声明外部符号（ABI 可选 `"C"`、`"Rust"` 等），声明以分号结尾；导出符号给外部链接时给函数加 `#[no_mangle]`（取消名称修饰）与 `#[link_name = ".."]`（改链接名）。这是 Rust 与 C/C++ 互操作的接口层：

```
mod foo {
    #[no_mangle]
    pub extern "Rust" fn my_demo_function(a: u32) -> u32 { a + 1 }
    // #[link_name] 给同一符号起别名
}

extern "Rust" {
    fn my_demo_function(a: u32) -> u32;           // 引用模块内导出的符号
    #[link_name = "my_demo_function"]
    fn my_demo_function_alias(a: u32) -> u32;
}
```

unsafe 能做而安全代码不能做的五件事

1. 解引用裸指针；2. 调用 `unsafe` 函数；3. 访问/修改可变静态变量；4. 实现 `unsafe trait`；5. 访问 `union` 字段。

`unsafe` 块内依旧享受大部分检查（类型、`move` 语义）——它只解锁上述五项，因此面积应当尽可能小，并始终附带 SAFETY 说明。

第 15 章 Clippy 与代码风格

对应题单：clippy（3 题）。Clippy 是官方 lint 工具，`cargo clippy` 运行，提示比编译器更“懂惯用法”的问题。

- **clippy1: 不要手写数学常数。** 自己算的 3.14159 精度不够，应使用 `std::f32::consts` 里的 `PI` 等官方常数（`approx_constant` lint）；
- **clippy2: 对 Option 的 for 循环改用 if let**——`for` 会隐式调用 `into_iter`，语义上你只想要“有值就处理一次”；
- **clippy3:** 一组风格问题合集——多余的 `return`、用 `let mut x = 0;` 初始化可变变量、无意义的 `&vec[..]` 等。读懂每条 lint 的提示比背规则重要。
- 常用补充 lint：`clippy::redundant_clone` 找不必要的 `clone`，`clippy::pedantic` 提示更习惯的写法，`#![deny(clippy::all)]` 级别可设为 `deny` 进 CI 强制执行。

第 16 章 强化训练：数据结构与算法实战

对应题单：algorithm1-10。这组题把前面所有知识揉进十个经典数据结构，每题都有明确的考察意图，是检验综合能力的最好材料。

16.1 链表：NonNull 与双链表反转 (algorithm1-2)

题单的链表用 `NonNull<Node<T>>`（非空裸指针）而不是 `Box` 存节点——为的是同一节点能被 `prev/next` 双向持有而绕开所有权检查（代价是手动管理内存，属 `unsafe` 领域）。合并两个有序链表是典型的**双指针迭代**：

```
// algorithm1 思路骨架：_dummy 头 + 双指针尾插
fn merge(l1: LinkedList<T>, l2: LinkedList<T>) -> LinkedList<T>
where T: Ord + Copy {
    let mut result = LinkedList::new();
    let (mut a, mut b) = (l1.start, l2.start); // 两条游标
    while let (Some(x), Some(y)) = (a, b) {
        unsafe {
            if (*x.as_ptr()).val <= (*y.as_ptr()).val {
                result.push_back((*x.as_ptr()).val);
                a = (*x.as_ptr()).next;
            } else {
                result.push_back((*y.as_ptr()).val);
                b = (*y.as_ptr()).next;
            }
        }
    }
    // 把剩余段整体接上，最后返回 result
    result
}
```

双向链表反转 (algorithm2) 循环不变量只有一句：**每一步交换当前节点的 prev 与 next**，游标沿原 `next` 前进，走完时表头变表尾。

16.2 泛型排序 (algorithm3)

签名 `fn sort<T: Ord>(array: &mut [T])` 要求对任意可全序类型原地排序。归并/堆排需要额外空间或索引运算，插入排序最贴合该签名（加 `PartialOrd` 约束或用 `array.swap(i, j)` 比较交换）：

```
fn sort<T: Ord>(array: &mut [T]) {
    for i in 1..array.len() {
        let mut j = i;
        while j > 0 && array[j - 1] > array[j] { // 大数往后冒（插入排序）
            array.swap(j - 1, j);
            j -= 1;
        }
    }
}
```

16.3 二叉搜索树 (algorithm4)

节点类型 `Option<Box<TreeNode<T>>>` 完美体现第 11 章：**Box 定大小、Option 表有无**。插入用递归 + match 重接子树；中序遍历天然有序：

```
impl<T: Ord> BinarySearchTree<T> {
    fn insert(&mut self, value: T) {
        match &mut self.root {
            None => self.root = Some(Box::new(TreeNode::new(value))),
            Some(node) => node.insert(value),
        }
    }
}

impl<T: Ord> TreeNode<T> {
    fn insert(&mut self, value: T) {
        match value.cmp(&self.value) {           // Ordering 三态
            Ordering::Less =>
                match &mut self.left {
                    None => self.left = Some(Box::new(TreeNode::new(value))),
                    Some(n) => n.insert(value),
                },
            Ordering::Greater => { /* 同理走右子树 */ }
            Ordering::Equal => {}                // 已存在则忽略
        }
    }
}
```

16.4 BFS 与 DFS (algorithm5-6)

图的邻接表用 `Vec<Vec<usize>>`，访问顺序用队列（`VecDeque`，BFS）或递归栈（DFS）维护，已访问集合用 `HashSet`。两者骨架只差容器选择：

```
use std::collections::{VecDeque, HashSet};

fn bfs_with_return(&self, start: usize) -> Vec<usize> { // algorithm5
    let mut visited = HashSet::new();
    let mut visit_order = Vec::new();
    let mut queue = VecDeque::new();
    visited.insert(start);
    queue.push_back(start);
    while let Some(v) = queue.pop_front() {           // 队列：先进先出 -> 广度优先
        visit_order.push(v);
        for &nei in &self.adj[v] {
            if visited.insert(nei) {                  // insert 返回 bool，一步判重
                queue.push_back(nei);
            }
        }
    }
    visit_order
}
```

```
fn dfs_util(&self, v: usize, visited: &mut HashSet<usize>,
           order: &mut Vec<usize>) { // algorithm6
    if visited.insert(v) {
        order.push(v); // 递归: 深入 -> 深度优先
        for &nei in &self.adj[v] {
            self.dfs_util(nei, visited, order);
        }
    }
}
```

16.5 栈与队列 (algorithm7-8)

algorithm7 用 Vec 实现栈完成**括号匹配**：左括号入栈，右括号弹栈比对；扫描结束栈非空则不匹配。核心循环：

```
fn bracket_match(s: &str) -> bool {
    let mut stack: Vec<char> = Vec::new();
    for c in s.chars() {
        match c {
            '(' | '[' | '{' => stack.push(c),
            ')' if stack.pop() != Some('(') => return false,
            ']' if stack.pop() != Some '[' => return false,
            '}' if stack.pop() != Some('{') => return false,
            _ => {}
        }
    }
    stack.is_empty() // 剩余未闭合的左括号 = 不匹配
}
```

algorithm8 **用两个队列实现栈**：push 进主队列；pop 时把前 n-1 个元素搬到辅助队列，取出最后一个，再交换两队列角色。题面给的 Queue 基于 Vec（dequeue 是 `Vec::remove(0)` 的 $O(n)$ 操作），重点在模拟思路而非性能。

16.6 二叉堆 (algorithm9)

二叉堆用数组存储，**父子下标关系**是全部核心（题面索引从 1 开始）：节点 i 的父节点是 $i / 2$ ，左右孩子是 $2i$ 与 $2i + 1$ 。`sift_up` 上浮（与父比较、交换直到满足序），`sift_down` 下沉（与较优孩子交换）。插入 = 尾部追加 + 上浮；取堆顶 = 换出根 + 下沉。

```
pub fn add(&mut self, value: T) {
    self.items.push(value);
    self.count += 1;
    self.sift_up(self.count); // 新元素从底部上浮
}

fn sift_up(&mut self, mut idx: usize) {
    while idx > 1 { // 根在 1 号位, 0 号是哨兵
        let parent = idx / 2;
        if (self.comparator)(&self.items[idx], &self.items[parent]) {
            self.items.swap(idx, parent); // 比较器决定大根/小根堆
            idx = parent;
        }
    }
}
```

```
    } else {
        break;
    }
}
}
```

16.7 无向图与 trait 抽象 (algorithm10)

最后一题把图抽象成 trait (`trait Graph`)，邻接表用 `HashMap<String, Vec<(String, i32)>>`。加边要**双向**插入且不产生重复边；`contains/edges` 方法从邻接表查询，节点不存在时返回自定义错误 `NodeNotInGraph`（实现 `Display`）。这是"trait 定义接口 + 结构体存数据 + Result 表达失败"的全套综合演练。

附录 A 题单与知识点对照表

按题单顺序列出每道（组）练习对应的核心知识点与本笔记章节，方便按题索引复习。

题单（题数）	核心知识点	对应章节
intro1-2 (2)	rustlings 流程、println! 格式化占位	1.5
variables1-6 (6)	let、类型推断、mut、shadowing、const	1.1
functions1-5 (5)	签名标注、表达式 vs 语句、返回值	1.3
if1-3 (3)	if 表达式、分支同型	1.4
quiz1 (1)	以上综合	1
primitive_types1-6 (6)	标量、数组 [T:N]、切片、元组解构	1.2
vecs1-2 (2)	Vec 两种创建、迭代与解引用修改	2.4
move_semantics1-6 (6)	所有权移动、&/&mut 借用、NLL	2.1-2.2
structs1-3 (3)	三种结构体、方法、关联函数	3.1
enums1-3 (3)	变体携带数据、match 解构	3.2
strings1-4 (4)	String/&str、deref 强制转换、字符串方法	2.3
modules1-3 (3)	pub、use、pub use 重导出、嵌套路径/glob	4
hashmaps1-3 (3)	HashMap、entry/or_insert、按旧值更新	5
quiz2 (1)	以上综合	1-5
options1-3 (3)	Some/None、if let/while let、ref 模式	6.1
errors1-6 (6)	Result、?、main 返回 Result、map_err、Box<dyn Error>	6.2-6.4

题单 (题数)	核心知识点	对应章节
generics1-2 (2)	泛型函数/结构体/impl	7.1
traits1-5 (5)	trait 定义、默认实现、impl Trait、多重约束	7.2
quiz3 (1)	trait bound + Display 综合运用	7.2
lifetimes1-3 (3)	生命周期标注、结构体生命周期、'static	7.3
tests1-4 (4)	assert!/assert_eq!/should_panic	9.1
iterators1-5 (5)	iter/iter_mut/into_iter、map/collect/fold	10
box1 (1)	Box 与递归类型	11.1
rc1 (1)	Rc 引用计数	11.2
arc1 (1)	Arc 跨线程共享	11.2、12.2
cow1 (1)	Cow 写时克隆	11.3
threads1-3 (3)	spawn/join、Arc<Mutex>、mpsc 通道	12
macros1-4 (4)	macro_rules!、宏导出、arm 分号	13
clippy1-3 (3)	数学常数、for-Option 改 if let、风格 lint	15
using_as 等 5 题	as、From/Into、FromStr、TryFrom、AsRef/AsMut	8
tests5-6 (2)	unsafe 契约、Box::into_raw/from_raw	14.1-14.2
tests7-8 (2)	build.rs、cargo:rustc-env / rustc-cfg	9.2
tests9 (1)	extern 块、#[no_mangle]、#[link_name]	14.3
algorithm1-2 (2)	NonNull 链表、有序合并、双链表反转	16.1
algorithm3 (1)	泛型排序	16.2
algorithm4 (1)	二叉搜索树 Option<Box<Node>>	16.3
algorithm5-6 (2)	BFS (VecDeque) 、DFS (HashSet/递归)	16.4
algorithm7-8 (2)	栈括号匹配、两队列实现栈	16.5
algorithm9 (1)	二叉堆上浮/下沉	16.6
algorithm10 (1)	trait 抽象无向图、邻接表	16.7

表 A-1 题单与知识点对照 (强化训练部分详见正文第 16 章)

附录 B 常见编译错误速查

做题时最常撞上的报错与一句话定位法。

错误信息 (片段)	含义与修法
cannot find value / function in this scope	名字未定义或未 use 引入；检查拼写与作用域
mismatched types	类型不一致：检查字面量后缀、函数签名、if 各分支类型
expected `i32`, found `&String` 类	少了 * 解引用或 .clone(); 或该传 &x 却传了 x
cannot assign twice to immutable variable	声明处加 mut
borrow of moved value: `x`	x 的所有权已被移走：改传 &x，或先 clone，或调整调用顺序
cannot borrow `x` as mutable because it is also borrowed as immutable	可变与不可变引用共存：收窄作用域，让读写不重叠
cannot borrow as mutable, as it is not declared as mutable	错误在绑定处：let mut x
use of moved value / use after move	Copy 类型没问题，堆类型被 move 后再用；clone 或借用
recursive type has infinite size	递归类型字段用 Box 包起来
this function takes N arguments but M were supplied	调用与签名参数个数不匹配
mismatched lifetime / returns a value referencing data owned by current fn	返回引用指向函数内局部数据：返回所有权 (String) 或改参数生命周期
the trait `Display` is not implemented	派生 #[derive(Debug)] 用 {:?}，或手动实现 Display
cannot borrow data in an `Arc` as mutable	Arc 内部不可变：配 Mutex/RefCell
`?` couldn't convert the error / the trait `From<...>` is not implemented	错误类型不匹配：map_err 转换，或统一 Box<dyn Error>

表 B-1 高频报错速查

附录 C 延伸学习资源

- **The Rust Programming Language** ("The Book", 官方教程)：本笔记每个主题对应的深读章节都可在 doc.rust-lang.org/book 找到；
- **Rust by Example** (doc.rust-lang.org/rust-by-example)：按例子学，适合配合题单交叉查阅；
- **Rustlings 官方仓库** (github.com/rust-lang/rustlings)：题单本体，README 有完整运行方式 (rustlings watch)；
- **std 标准库文档** (doc.rust-lang.org/std)：做题时养成查 std 习惯，Iterator/Option/Result 的方法列表值得通读一遍；
- **The Little Book of Rust Macros** (veykril.github.io/tlborm)：第 13 章之后的进阶；
- **Rust Nomicon** (doc.rust-lang.org/nomicon)：unsafe 与健全性的深入材料，对应第 14 章；
- **Jon Gjengset 的 Crust of Rust 系列**：进阶主题视频生命周期、迭代器、智能指针等进阶主题的专讲视频合集。